TRANSIT

Gestion d’un entrepôt logistique

Rapport de projet BD70 · Système de gestion d’un entrepôt logistique

Eliott LE · Semih TASKIN · Thomas VUILLIN

UTBM · BD70 Conception et administration des bases de données · Automne 2026

Table des matières

Introduction	3
Cahier des charges	4
Cas d’utilisation	5
Modèle conceptuel de données	8
Schéma relationnel	14
Normalisation	20
Implémentation de la base	21
Fonctionnalités du site et des quais	23
Fonctionnalités de réception, de stock et d’expédition	28
Vérification	34
Données de démonstration et performances	35
Conclusion	37

Introduction

Lorsqu'un client signale un problème sur un lot de café, l'entrepôt doit pouvoir répondre à trois questions. Qui a livré ce lot ? Où la marchandise a-t-elle été rangée ? Dans quel camion est-elle repartie ? Le stock disponible à l'instant présent ne suffit pas à répondre. Il faut conserver les événements qui expliquent comment les quantités ont évolué.

Le sujet demande de concevoir une base de données relationnelle pour les opérations quotidiennes d'un entrepôt : flux de camions, quais, transporteurs, réception, stockage, préparation et expédition, avec une traçabilité complète et des indicateurs de pilotage. Nous avons développé **Transit**, une application web en PHP (PDO) adossée à MariaDB, pour un entrepôt unique.

Ce rapport suit la démarche du cours : analyse des besoins et cas d'utilisation, modèle conceptuel entité-association, traduction en schéma relationnel, normalisation, implémentation (contraintes, vues, procédures, déclencheurs, évènements, transactions, droits), puis présentation des fonctionnalités et analyse des performances.

Cahier des charges

Contexte. Un entrepôt logistique reçoit des camions de fournisseurs et expédie les commandes de ses clients propriétaires de la marchandise. Les camions se présentent au poste de garde, avec ou sans rendez-vous, puis sont appelés à un quai compatible. Les marchandises reçues sont contrôlées, rangées par lot dans des emplacements, puis prélevées pour constituer des colis et palettes chargés dans un camion sortant.

Objectif. Fournir une base de données normalisée couvrant l'ensemble de ces opérations, une interface web pour les manipuler, et des indicateurs : occupation des quais, temps d'attente et d'immobilisation par transporteur, remplissage par zone, taux d'écart en réception, productivité des préparateurs, articles proches de leur date limite.

Domaine du sujet	Besoins retenus	Interface
Authentification	Inscription d'un compte en attente, activation par l'administrateur, connexion, cinq profils aux droits distincts.	Oui (obligatoire)
Transporteurs, chauffeurs, véhicules	Sociétés, contacts, contrats, chauffeurs et habilitations (matières dangereuses), véhicules (type, capacité en kg et en palettes, équipements).	Oui
Quais et rendez-vous	Quais (type, équipements, types de véhicules acceptés), rendez-vous sans chevauchement et compatibles, indisponibilités, état libre / réservé / occupé / hors service.	Oui (obligatoire)
Flux de camions	Arrivée, contrôle documentaire, parking, appel à quai, début et fin d'opération, départ, horodatés ; arrivées sans rendez-vous ; temps d'attente et retards.	Oui (obligatoire)
Réception	Avis d'expédition, quantités reçues, lots, dates limites, état des palettes, écarts et réserves, rangement.	Oui (obligatoire)
Emplacements et stocks	Zones, allées, racks, niveaux, emplacements et capacités ; catalogue d'articles et propriétaires ; stock par article, lot et emplacement ; historique des mouvements ; compatibilité produit / zone.	Oui (obligatoire)
Préparation et expédition	Commandes et lignes, listes de prélèvement FEFO affectées à un préparateur, colis et palettes, chargement, bon de livraison et lettre de voiture.	Oui
Traçabilité et tableaux de bord	Historique complet d'un lot ; indicateurs de pilotage.	Oui

Contraintes techniques. SGBD MariaDB 10.4 (moteur InnoDB, utf8mb4) ; base en troisième forme normale ; données factices en quantité ; sécurité et confidentialité (mots de passe hachés, requêtes préparées, moindre privilège) ; optimisation des requêtes ; automatisation interne à la base, notamment l'historisation.

Périmètre. Un seul site. Une commande est expédiée en une seule fois. Les indicateurs d'activité portent sur une période choisie ; le stock et les alertes de date limite reflètent la situation courante.

Cas d’utilisation

Une première version du diagramme (V1) a été relue par l’enseignant. Ses remarques portaient sur quatre points : l’héritage entre acteurs (le responsable n’est pas un préparateur, l’administrateur cumulait trop de rôles), l’absence du transporteur, la répartition des rôles sur les commandes, et des cas trop globaux à éclater en sous-cas reliés entre eux.

La version présentée ici (V2) y répond :

- **aucun héritage** : chaque acteur est dessiné une seule fois et relié uniquement aux cas qu’il réalise ; l’administrateur ne gère que les comptes et le journal ;
- **le transporteur** est ajouté, limité aux rendez-vous de sa société ;
- **les commandes** sont réparties : le responsable gère les commandes et affecte un préparateur ; le préparateur prépare et expédie uniquement les commandes qui lui sont affectées ;
- **les quatre cas globaux sont éclatés** en sous-cas reliés par « include » et « extend », sur la même page : par exemple, « Enregistrer l’arrivée d’un camion » inclut « Contrôler les documents d’entrée », et « Traiter une arrivée sans rendez-vous » l’étend sous la condition [aucun rendez-vous] ; « Préparer une commande affectée » inclut la réservation FEFO, le prélèvement et la constitution des colis.

Conformément au sujet, la planification des quais revient au responsable d’exploitation, et l’agent d’accueil tient le poste de garde.

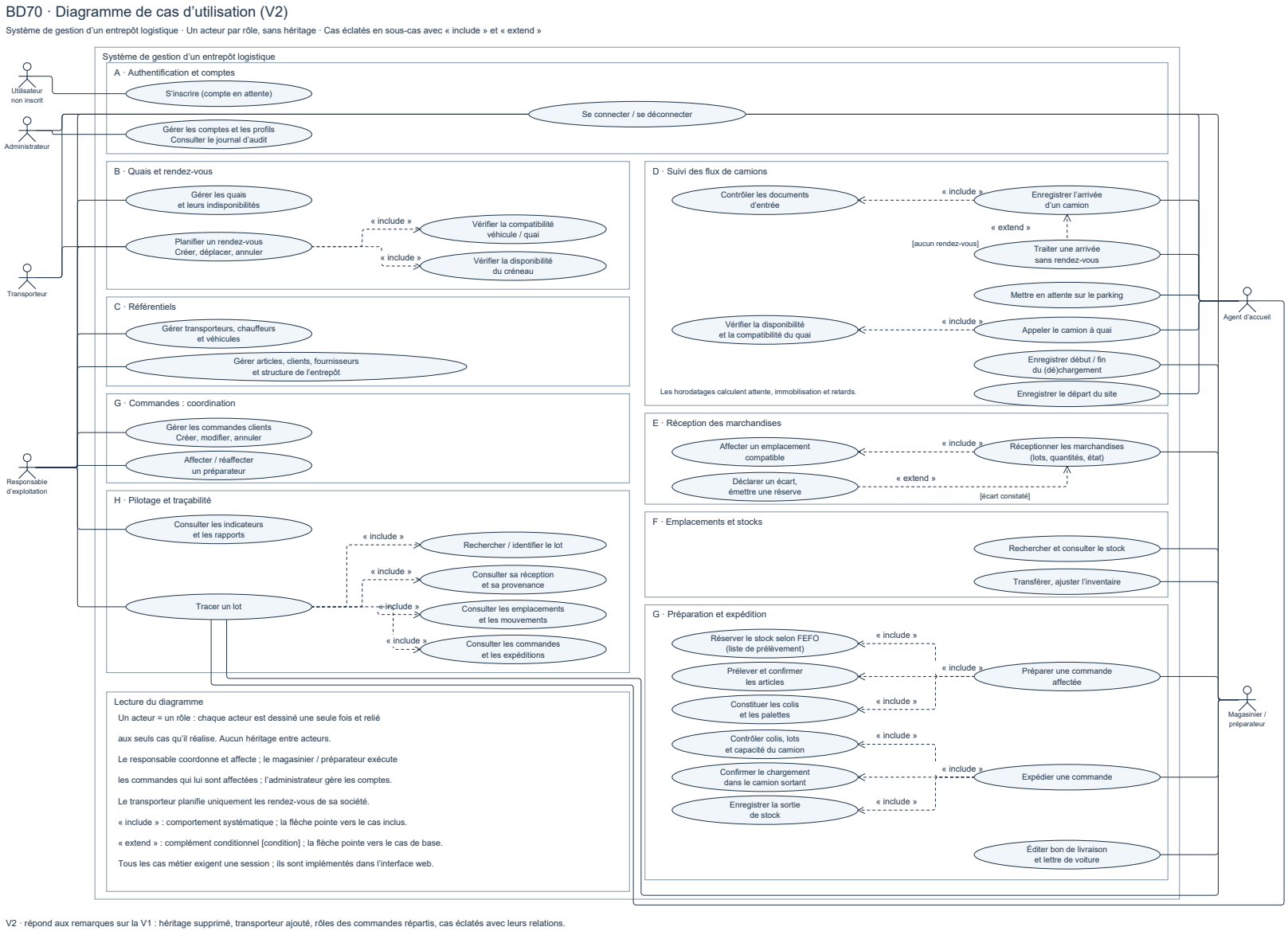


Figure 1 – Diagramme de cas d'utilisation V2, vue d'ensemble : un acteur par rôle, sans héritage.

Acteur	Responsabilité dans Transit
Administrateur	Active les comptes, attribue les profils et consulte le journal ; aucun accès aux données métier.
Responsable d’exploitation	Tient les référentiels, planifie les quais, gère les commandes, affecte les préparateurs et consulte les indicateurs.
Agent d’accueil	Enregistre l’arrivée (avec ou sans rendez-vous), le contrôle documentaire, l’attente sur le parking, l’appel à quai et le départ.
Magasinier / préparateur	Horodate le début et la fin des opérations à quai, réceptionne, range, transfère, et prépare uniquement les commandes qui lui sont affectées.
Transporteur	Demande, modifie ou annule les rendez-vous de sa propre société, avec ses chauffeurs et véhicules.

Chaque cas correspond à un droit vérifié côté serveur (matrice détaillée dans docs/matrice-crud-droits.md). Le cours rappelle qu’il est inutile d’enregistrer des données qu’aucun cas n’utilise : chaque entité du modèle conceptuel est alimentée ou consultée par au moins un de ces cas.

Modèle conceptuel de données

La première version du modèle entité-association mélangeait deux formalismes (attributs listés dans les rectangles, associations en losanges) et avait été rendue en image PNG. L'enseignant a relevé qu'elle « ne respecte pas un formalisme précis » et demandé un format vectoriel. Le modèle est donc refait entièrement en notation de Chen, celle du cours : entités en rectangles, associations en losanges, attributs en ovales, identifiants soulignés, attributs propres à une association en ovales reliés au losange, cardinalités min,max sur chaque trait. Comme dans le cours (« Client 1 — Possède — 0,N Facture »), la cardinalité écrite du côté d'une entité indique combien de fois l'autre entité peut lui être liée : entre TRANSPORTEUR et CHAUFFEUR, on lit « 1,1 » côté transporteur (un chauffeur appartient à une seule société) et « 0,N » côté chauffeur (une société emploie 0 à N chauffeurs). Il compte 35 entités et 58 associations : 50 associations 1:N, 5 associations N:N et 3 associations 1:1. Deux associations portent un attribut propre : la date de validité d'une habilitation de chauffeur et la quantité affectée d'un prélèvement à un colis.

Le modèle complet est fourni en PDF et SVG vectoriels (texte et formes, aucune image, à lire avec zoom) et en source Draw.io modifiable. Pour la lecture, les trois figures suivantes en sont des extraits exacts par domaine : mêmes entités, attributs et cardinalités, seule la disposition change.

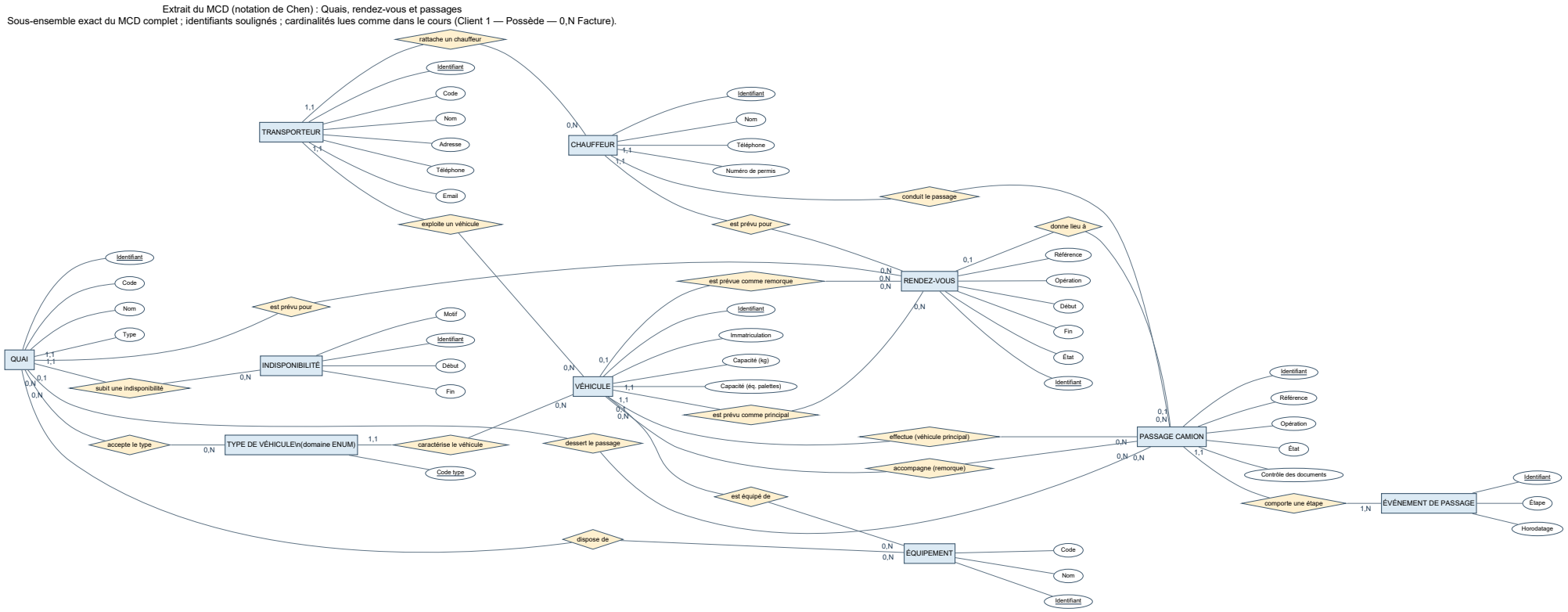


Figure 2 – Extrait du MCD : quais, rendez-vous et passages.

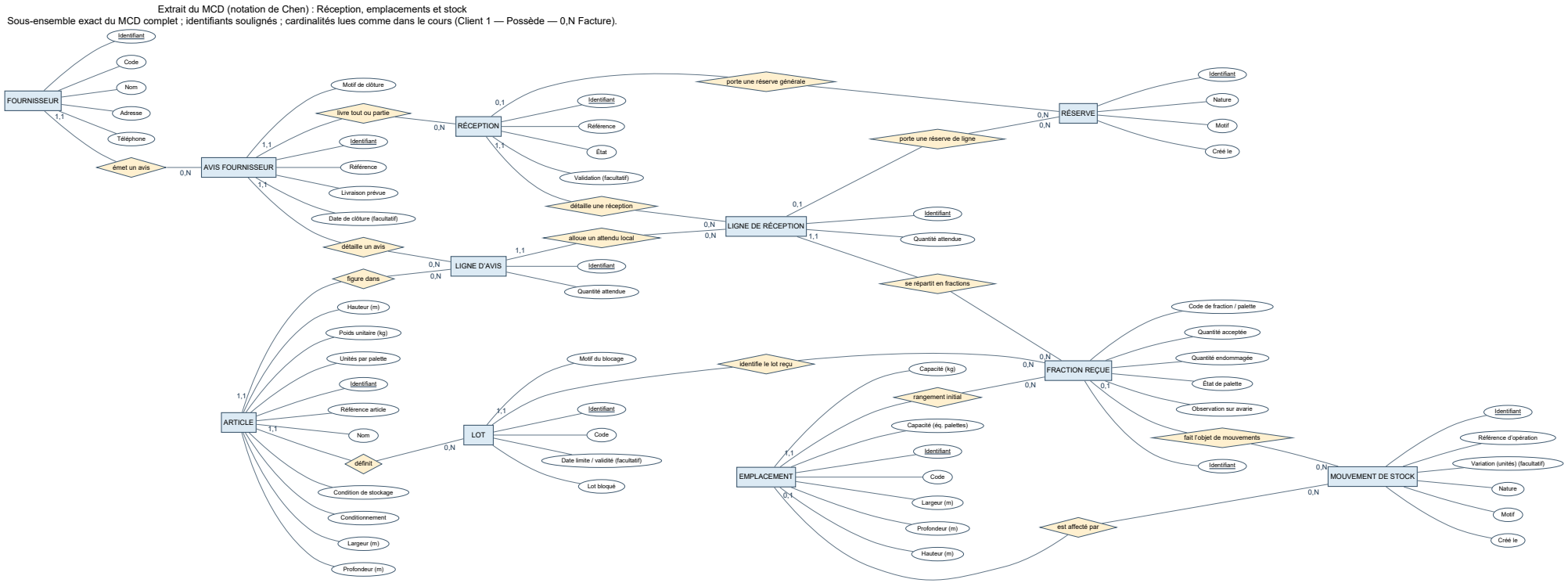


Figure 3 – Extrait du MCD : réception, emplacements et stock.

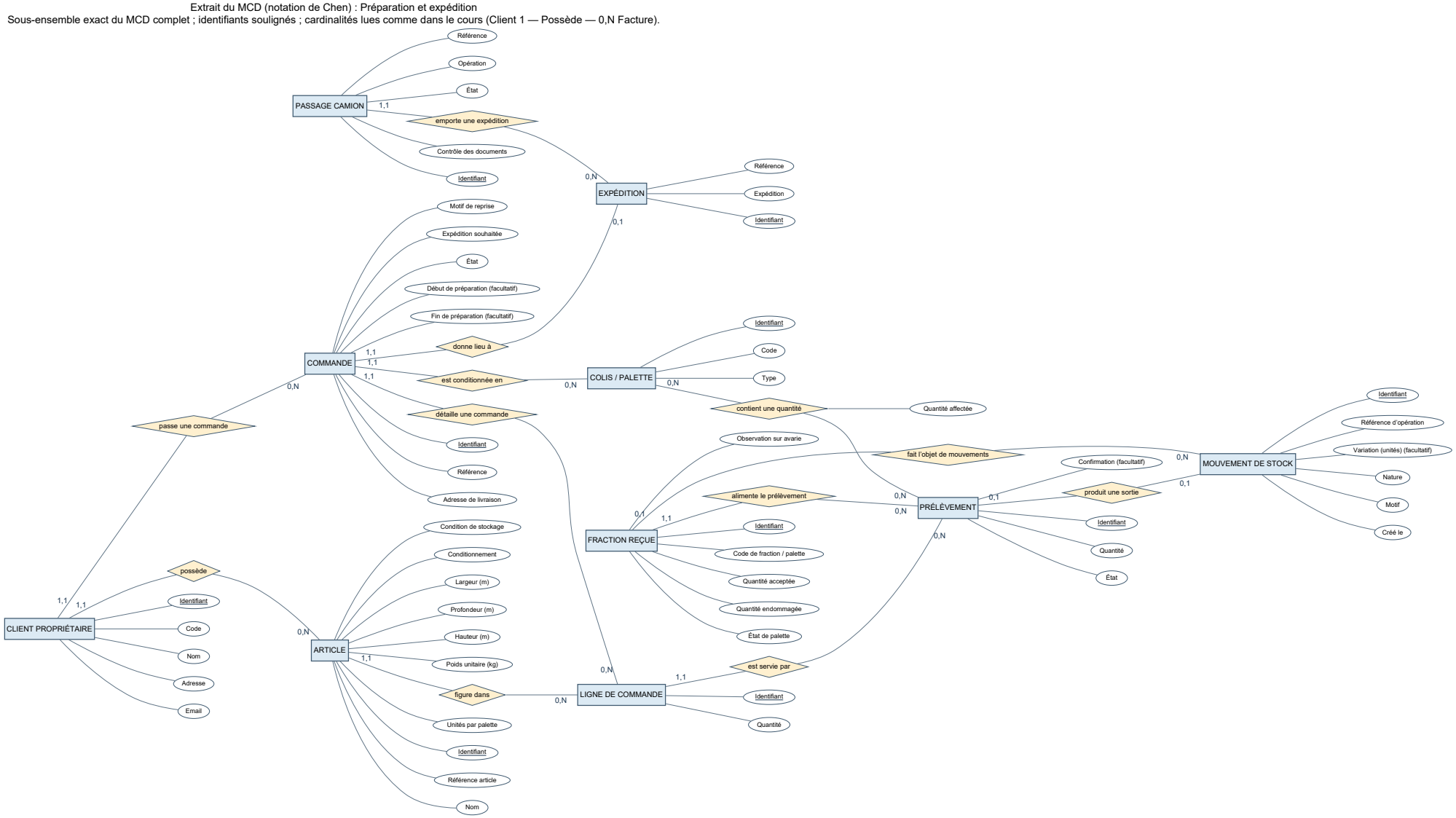


Figure 4 – Extrait du MCD : préparation et expédition.

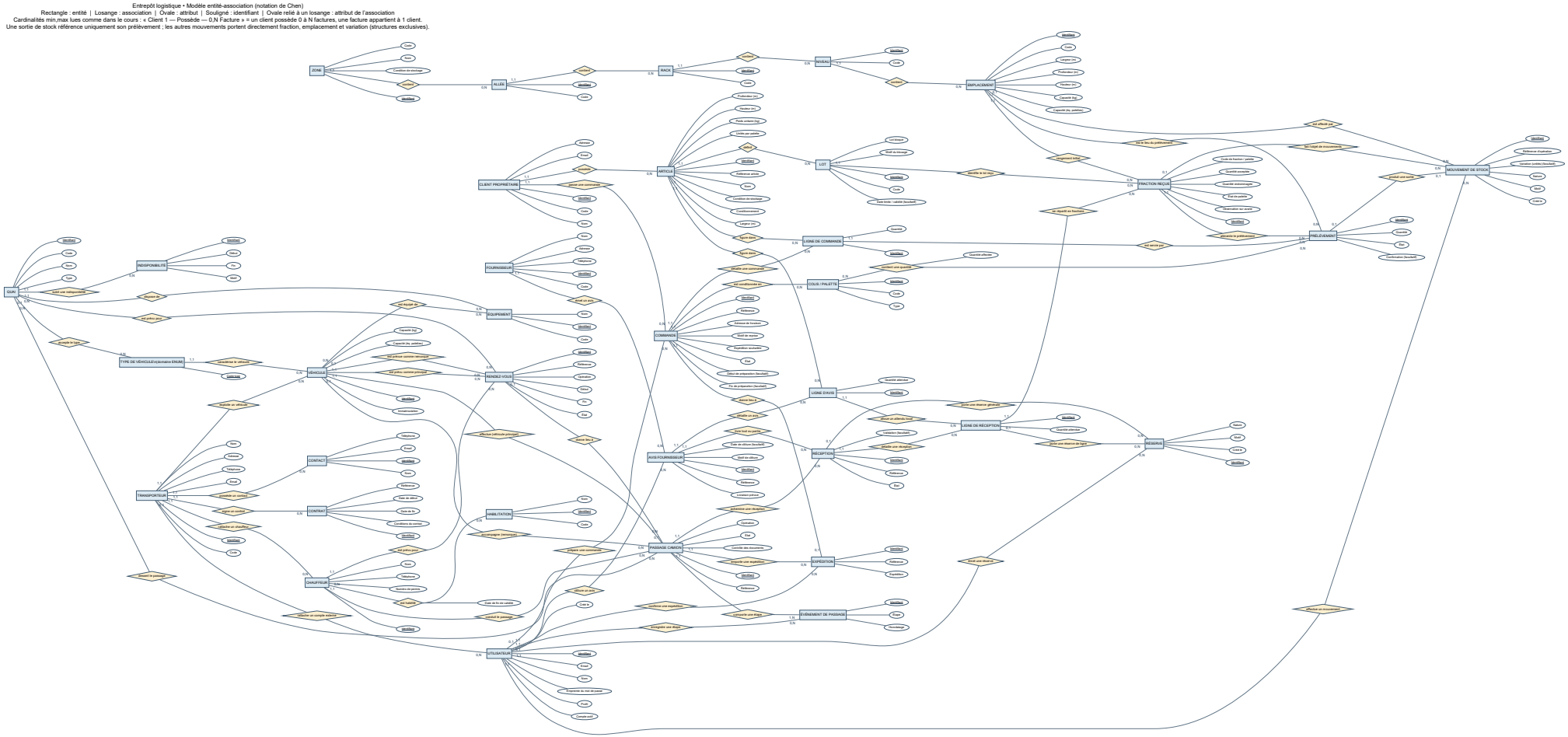


Figure 5 – MCD complet (document vectoriel : zoomer pour lire).

Distinguer la prévision du fait constaté. Le rendez-vous décrit le quai et les ressources prévus. Le passage décrit le camion effectivement arrivé et la chronologie de ses opérations. Une réservation peut rester sans passage et un camion peut arriver spontanément : l'association porte donc 0,1 des deux côtés. De même, un avis fournisseur décrit les quantités attendues, alors que la réception conserve le comptage réel et les réserves.

Conserver la provenance. Un même lot peut arriver en plusieurs livraisons. Chaque fraction reçue conserve son lien avec la ligne de réception, le lot et l'emplacement initial. Les transferts expliquent ses déplacements ultérieurs. Le prélèvement relie cette fraction à une ligne de commande et à l'emplacement où elle a été prise.

Placer l'attribut au bon endroit. Si un prélèvement de douze unités est réparti entre deux colis, les quantités sept et cinq ne dépendent ni du colis seul ni du prélèvement seul, mais du couple : la quantité est un attribut de l'association N:N « contient une quantité ».

Type de véhicule. Le type (tracteur, remorque, porteur, fourgon) est une entité du MCD car un quai accepte plusieurs types (association N:N). Ses valeurs étant fixées par le sujet, il est implémenté par un domaine ENUM, et l'association par la table dock_vehicle_type.

Schéma relationnel

Règles de passage appliquées (cours, méthodologie de conception).

- Une entité devient une table, un attribut une colonne, l'identifiant la clé primaire.
- Une association 1:N devient une clé étrangère du côté N. Exemple : driver.carrier_id référence carrier.id (une société emploie plusieurs chauffeurs, un chauffeur appartient à une société).
- Une association N:N devient une table de jointure portant les deux clés étrangères, l'unicité du couple et les attributs de l'association : driver_qualification, vehicle_equipment, dock_equipment, dock_vehicle_type et package_pick (avec la quantité affectée).
- Une association 1:1 est traduite par une clé étrangère UNIQUE dans la table dont l'existence dépend de l'autre : truck_visit.appointment_id, shipment.order_id et stock_movement.pick_id. Les deux entités restent séparées car elles ont des cycles de vie distincts (un rendez-vous peut exister sans passage).

Les tables de jointure ont une clé artificielle id en plus de la contrainte UNIQUE sur le couple : la clé candidate composée reste garantie, et l'interface générique de saisie manipule toutes les tables de la même façon.

Le schéma compte 43 tables : 34 issues des entités (le type de véhicule est un domaine ENUM), 5 tables de jointure, 3 tables techniques (journal d'audit, verrou du site et soldes de stock stock_balance) et la table d'historique appointment_archive (voir Implémentation de la base). Il comprend 243 colonnes, 68 clés étrangères, 38 contraintes UNIQUE et 22 contraintes CHECK.

Notation : clé primaire soulignée, # clé étrangère (→ table référencée), [U] unique, [NULL] facultatif ; les autres colonnes sont NOT NULL.

aisle (id, #zone_id → zone, code [U])

appointment (id, reference [U], #dock_id → dock, #driver_id → driver, #vehicle_id → vehicle, #trailer_id → vehicle [NULL], operation, starts_at, ends_at, status)

appointment_archive (id, reference [U], #dock_id → dock, #driver_id → driver, #vehicle_id → vehicle, #trailer_id → vehicle [NULL], operation, starts_at, ends_at, status, archived_at)

article (id, #owner_id → owner, sku, name, storage, packaging, width_m, depth_m, height_m, unit_kg, units_per_pallet) ; UNIQUE(owner_id, sku)

audit_log (id, entity, entity_id, operation, #actor_id → user_account [NULL], happened_at, details)

carrier (id, code [U], name, address, phone, email)

carrier_contact (id, #carrier_id → carrier, name, phone, email)

carrier_contract (id, #carrier_id → carrier, reference [U], starts_on, ends_on, terms)

customer_order (id, reference [U], #owner_id → owner, destination, revision_note, desired_on, status, #preparer_id → user_account [NULL], picking_at [NULL], ready_at [NULL])

dock (id, code [U], name, type)

dock_downtime (id, #dock_id → dock, starts_at, ends_at, reason)

dock_equipment (id, #dock_id → dock, #equipment_id → equipment) ; UNIQUE(dock_id, equipment_id)

dock_vehicle_type (id, #dock_id → dock, vehicle_type) ; UNIQUE(dock_id, vehicle_type)

driver (id, #carrier_id → carrier, name, phone, license_number [U])

driver_qualification (id, #driver_id → driver, #qualification_id → qualification, expires_on) ; UNIQUE(driver_id, qualification_id)

equipment (id, code [U], name)

location (id, #level_id → rack_level, code [U], width_m, depth_m, height_m, max_kg, max_pallets)

lot (id, #article_id → article, code, expires_on [NULL], blocked, block_reason) ; UNIQUE(article_id, code)

notice_line (id, #notice_id → supplier_notice, #article_id → article, expected_qty) ; UNIQUE(notice_id, article_id)

order_line (id, #order_id → customer_order, #article_id → article, qty) ; UNIQUE(order_id, article_id)

owner (id, code [U], name, address, email)

package (id, #order_id → customer_order, code [U], type)

package_pick (id, #package_id → package, #pick_id → pick, qty) ; UNIQUE(package_id, pick_id)

pick (id, #order_line_id → order_line, #receipt_item_id → receipt_item, #location_id → location, qty, status, confirmed_at [NULL])

qualification (id, code [U], name)

rack (id, #aisle_id → aisle, code [U])

rack_level (id, #rack_id → rack, code [U])

receipt (id, reference [U], #notice_id → supplier_notice, #visit_id → truck_visit, status, validated_at [NULL])

receipt_item (id, #line_id → receipt_line, #lot_id → lot, pallet_code [U], accepted_qty, damaged_qty, pallet_state, reservation, #location_id → location)

receipt_line (id, #receipt_id → receipt, #notice_line_id → notice_line, expected_qty) ; UNIQUE(receipt_id, notice_line_id)

receipt_reserve (id, #receipt_id → receipt [NULL], #line_id → receipt_line [NULL], kind, reason, #actor_id → user_account, created_at)

shipment (id, #order_id → customer_order [U], #visit_id → truck_visit, reference [U], shipped_at, #actor_id → user_account)

site_lock (id, label)

stock_balance (#receipt_item_id → receipt_item, #location_id → location, qty, reserved_qty)

stock_movement (id, operation_ref, #receipt_item_id → receipt_item [NULL], #location_id → location [NULL], delta [NULL], kind, #pick_id → pick [U] [NULL], reason, #actor_id → user_account, created_at)

supplier (id, code [U], name, address, phone)

supplier_notice (id, reference [U], #supplier_id → supplier, expected_on, closed_at [NULL], #closed_by → user_account [NULL], closure_note)

truck_visit (id, reference [U], #appointment_id → appointment [U] [NULL], #driver_id → driver, #vehicle_id → vehicle, #trailer_id → vehicle [NULL], operation, #dock_id → dock [NULL], status, document_note)

user_account (id, email [U], name, password_hash, role, active, created_at, #carrier_id → carrier [NULL])

vehicle (id, #carrier_id → carrier, plate [U], type, max_kg, max_pallets)

vehicle_equipment (id, #vehicle_id → vehicle, #equipment_id → equipment) ; UNIQUE(vehicle_id, equipment_id)

visit_event (id, #visit_id → truck_visit, state, happened_at, #actor_id → user_account) ; UNIQUE(visit_id, state)

zone (id, code [U], name, storage)

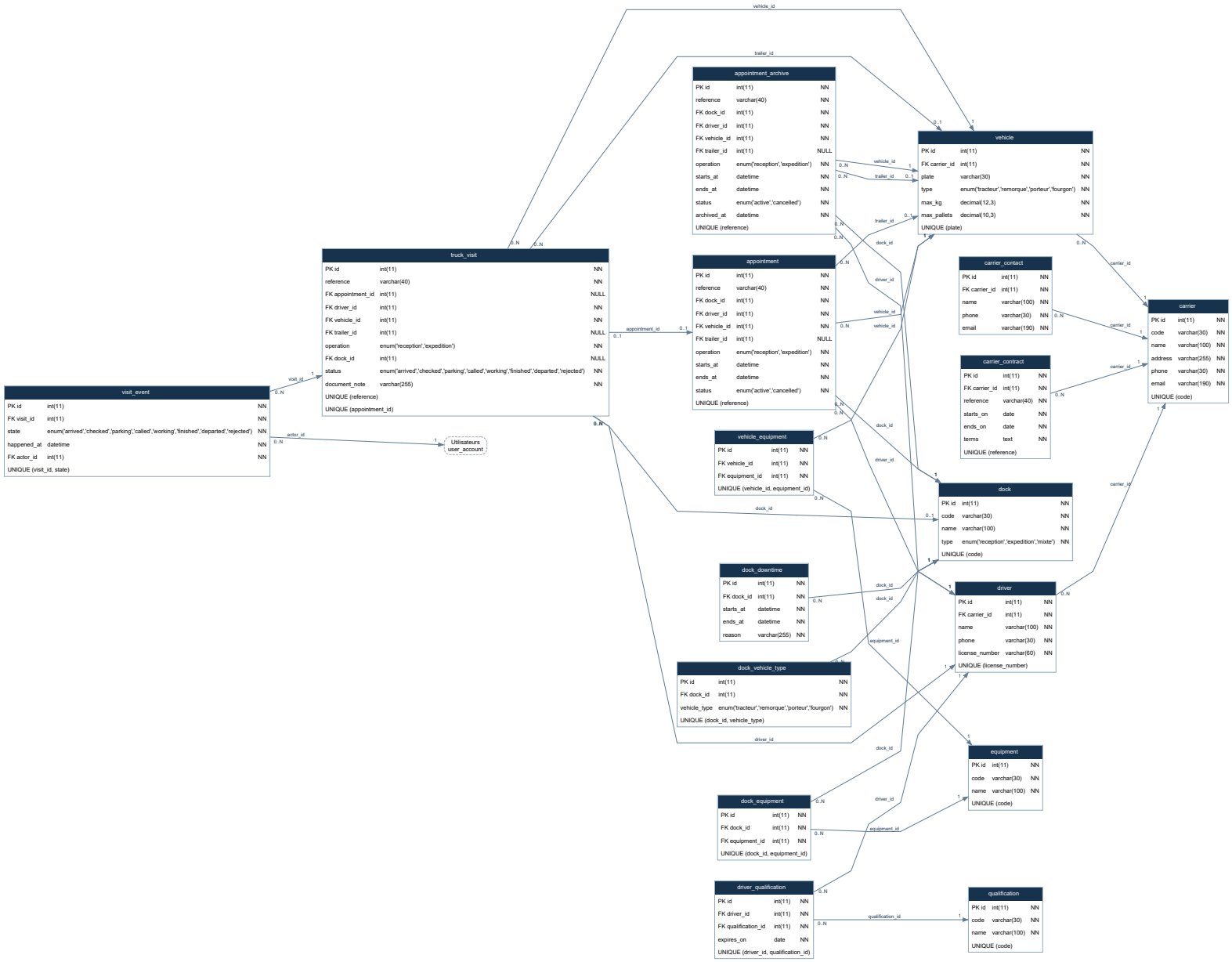


Figure 6 – Schéma relationnel : transport, quais, rendez-vous et passages.

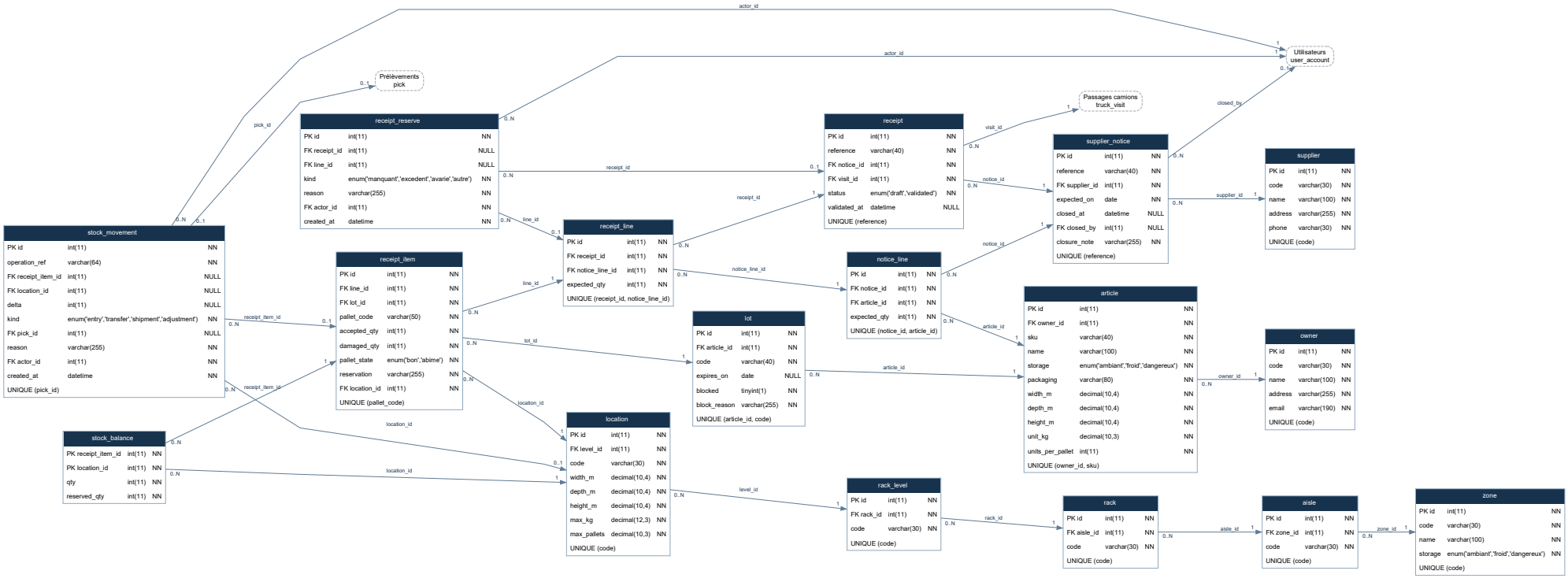


Figure 7 – Schéma relationnel : réception, emplacements et stock.

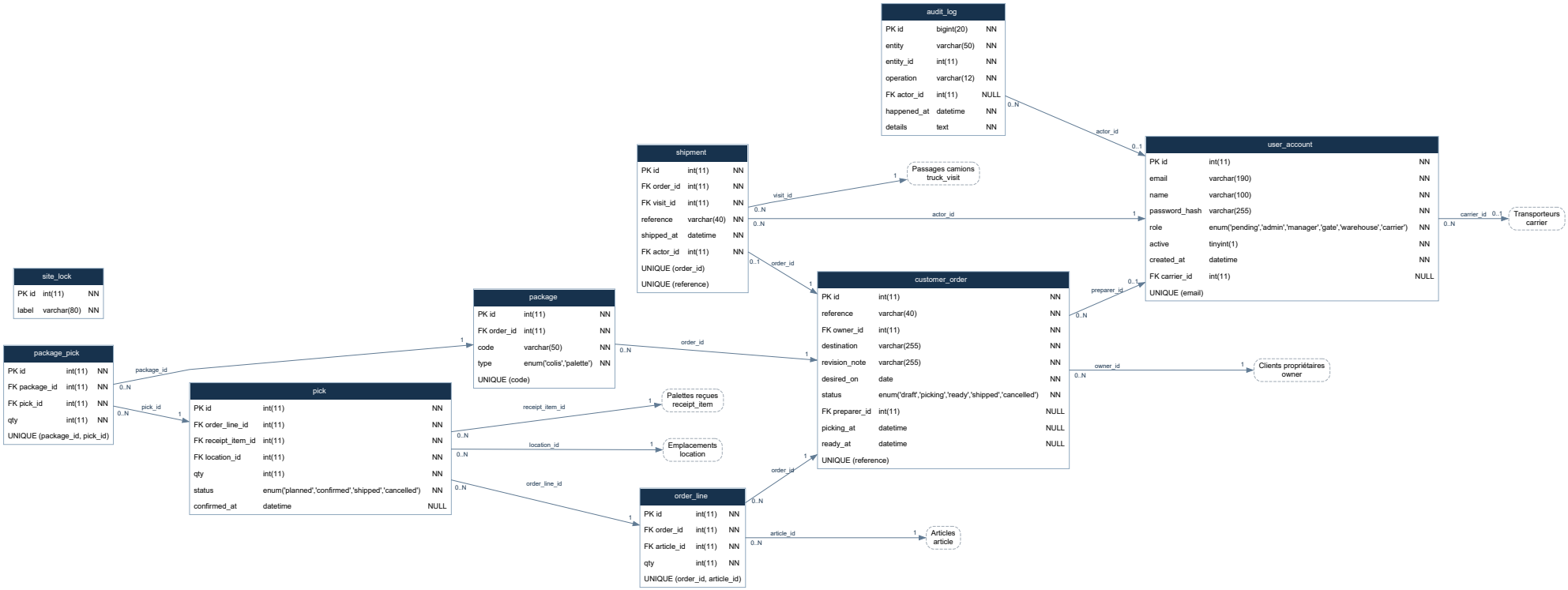


Figure 8 – Schéma relationnel : commandes, comptes et tables techniques.

Contraintes d’intégrité. Outre les clés, les CHECK traduisent des règles métier : fin après le début (rendez-vous, contrats, indisponibilités), quantités strictement positives, capacités positives, réserve obligatoire dès qu’une quantité est endommagée, profil transporteur rattaché à exactement une société et comptes internes sans société, structures exclusives d’un mouvement de stock (une sortie ne référence que son prélèvement ; les autres mouvements portent fraction, emplacement et variation). Aucune clé étrangère n’utilise ON DELETE CASCADE : une suppression qui ferait disparaître un historique est refusée.

Normalisation

Première forme normale. Chaque cellule contient une valeur atomique : les équipements d'un véhicule ou les habilitations d'un chauffeur ne sont pas des listes dans une colonne, mais des lignes de tables de jointure (l'erreur « Produits = 1, 2, 3 » du cours est évitée). Seul le journal audit_log stocke un instantané JSON avant/après : c'est une trace technique, jamais interrogée par les traitements métier.

Deuxième forme normale. Elle concerne les clés composées. Dans order_line, la clé candidate (order_id, article_id) détermine la quantité ; la désignation de l'article, qui ne dépend que de article_id, reste dans article. Même raisonnement pour notice_line, receipt_line et package_pick.

Troisième forme normale. Aucune colonne non clé n'en détermine une autre. Exemples : la zone d'un emplacement est obtenue par jointure (emplacement → niveau → rack → allée → zone) et n'est pas recopiée ; aucun total quantité × prix n'est stocké ; les temps d'attente sont calculés par une vue ; le solde de stock est conservé dans stock_balance pour la performance, comme redondance contrôlée (voir ci-dessous) ; une sortie de stock référence son prélèvement, qui détermine déjà la fraction, l'emplacement et la quantité (un déclencheur refuse toute valeur divergente).

Forme normale de Boyce-Codd. Les déterminants rencontrés sont des clés candidates (code, référence, immatriculation, couple de jointure) ; aucune dépendance d'un attribut non clé vers une partie de clé n'a été identifiée.

Dénormalisations assumées. Quatre choix sont volontaires et documentés (docs/normalisation-3nf.md) :

Colonne	Pourquoi elle n'est pas une redondance gênante
truck_visit.driver_id, vehicle_id, trailer_id, operation	Indispensables aux passages sans rendez-vous (27 % du jeu de démonstration). Pour un passage avec rendez-vous, ils sont recopiés à l'arrivée : c'est une dépendance conditionnelle assumée, sans anomalie de mise à jour car un rendez-vous utilisé ne peut plus être modifié. Le quai effectif (dock_id) est, lui, attribué à l'appel et peut différer du quai prévu.
truck_visit.status, customer_order.status, receipt.status	Projection de l'état courant, mise à jour dans la même transaction que l'événement correspondant, pour éviter de recalculer la dernière étape à chaque affichage.
receipt_line.expected_qty	Quantité attendue pour cette livraison : une ligne d'avis peut être livrée en plusieurs fois ; elle n'est donc pas déterminée par la ligne d'avis seule.
stock_balance.qty, reserved_qty	Solde et réservations par fraction et emplacement, déductibles de l'historique des mouvements et des prélèvements. Conservés pour la performance (voir Données de démonstration et performances) : seuls des déclencheurs les écrivent, dans la même transaction que le mouvement ou la réservation ; le compte applicatif n'a aucun droit d'écriture ; un contrôle compare chaque solde au recalcul depuis l'historique (v_balance_controle).

Implémentation de la base

Scripts. L'installation est reproductible sur une base vide : sql/01_schema.sql (tables et contraintes), 02_vues.sql, 03_procedures.sql, 04_triggers.sql, 05_index.sql, 08_historisation.sql, 09_optimisation.sql ; le script scripts/install.php les exécute dans l'ordre et refuse une base non vide. Les droits sont décrits dans 06_privileges.example.sql et les données factices dans 07_donnees_demo_riches.sql.

Vues. Onze vues isolent les calculs réutilisés : v_stock_movement (sorties projetées depuis leur prélèvement), v_balance et v_reserved (solde et réservations lus dans stock_balance), v_balance_controle (même solde recalculé par SUM … GROUP BY sur tout l'historique, pour le contrôle), v_stock (disponible = solde − réservations, LEFT JOIN et COALESCE), v_location (jointures de la structure physique), v_location_usage (occupation en kg, m³ et palettes), v_visit (chronologie d'un passage par LEFT JOIN sur ses événements, temps d'attente et d'immobilisation), v_receipt_discrepancy et v_notice_line_progress (écarts de réception), v_appointment_all (rendez-vous courants et historisés, UNION ALL).

Procédures stockées. lock_site verrouille la ligne du site (SELECT … FOR UPDATE) au début de chaque transaction d'écriture ; trace_lot renvoie l'historique paginé d'un lot (réception, mouvements, commande, expédition, camion) ; archive_appointments déplace les rendez-vous anciens vers l'historique.

Fonction stockée. stock_disponible(article, date) renvoie la quantité disponible d'un article à une date : lots non bloqués et non périmés, réservations déduites (CREATE FUNCTION … RETURNS INT READS SQL DATA). La préparation de commande l'appelle pour refuser d'emblée une demande supérieure au stock disponible, avant toute réservation.

Transactions. Toute opération qui touche plusieurs tables (valider une réception, transférer, préparer une commande, expédier) est exécutée dans une transaction : START TRANSACTION, appel de lock_site, contrôles, écritures, puis COMMIT ; à la moindre erreur, ROLLBACK annule toutes les écritures. Exemple : deux responsables réservent le même créneau. La première transaction prend le verrou, vérifie et enregistre ; la seconde attend, relit le planning, détecte le chevauchement et est annulée.

Déclencheurs. Le cours recommande de limiter les déclencheurs à la cohérence et à la performance. Ils sont regroupés en trois familles :

Famille	Nombre	Rôle
Cohérence métier	3	appointment_no_overlap_insert et _update refusent (SIGNAL) un chevauchement sur un quai même si l'écriture contourne l'application ; normalize_shipment refuse une sortie incohérente avec son prélèvement.
Protection de l'historique	11	Refus de UPDATE et DELETE sur les faits immuables : mouvements, événements de passage, expéditions, réserves, journal ; pick_shipped_frozen fige un prélèvement expédié.
Calcul incrémental (performance)	4	balance_movement_insert ajoute la variation de chaque mouvement au solde de stock_balance (INSERT … ON DUPLICATE KEY UPDATE, coût constant) ; balance_pick_insert, _update et _delete tiennent à jour les quantités réservées. Même principe que la table de statistiques hebdomadaires du TD Automatisation.
Journal d'audit	109	Un déclencheur AFTER INSERT, UPDATE ou DELETE par table métier écrit l'ancienne et la nouvelle valeur dans audit_log, avec l'auteur (sans jamais copier le mot de passe). Ils sont générés à partir du schéma (scripts/build_triggers.py) et ne font qu'un INSERT chacun.

Historisation et évènement planifié. Conformément au mécanisme présenté en cours, la procédure archive_appointments(p_days) déplace dans une transaction les rendez-vous terminés depuis p_days jours et n'ayant donné lieu à aucun passage (annulés ou non honorés) vers la table appointment_archive, puis les supprime de appointment : la table consultée par le planning reste légère. L'évènement ev_archive_appointments l'exécute chaque nuit avec un délai d'un an (CREATE EVENT … ON SCHEDULE EVERY 1 DAY, actif quand event_scheduler=ON). La vue v_appointment_all réunit les deux tables avec UNION ALL. Les mouvements de stock, eux, ne sont jamais modifiés : ils forment l'historique complet qui permet de

recalculer tout solde.

Index. Les clés primaires, UNIQUE et étrangères sont indexées automatiquement par InnoDB. Huit index composés ciblent les recherches fréquentes : créneaux d'un quai (dock_id, status, starts_at, ends_at), passages par état, événements par période, mouvements par fraction et emplacement, mouvements par date, lots pour FEFO, réservations, avis par date. Les index ont un coût en écriture : nous n'en avons pas créé d'autres sans mesure.

Sécurité. Les mots de passe sont stockés hachés (password_hash) ; toutes les requêtes sont préparées (PDO, émulation désactivée), ce qui protège contre l'injection SQL ; les formulaires portent un jeton CSRF et chaque action vérifie le profil et l'objet concerné côté serveur. Côté SGBD, le principe du moindre privilège s'applique : l'application se connecte avec un compte dédié, limité à 127.0.0.1, qui n'a aucun droit de structure, seulement SELECT et EXECUTE, et INSERT seul sur les historiques ; il n'a aucun droit d'écriture sur le journal, le verrou, l'historique des rendez-vous et les soldes de stock. Le script d'exemple crée aussi un compte de reporting limité à des vues et un compte de sauvegarde, et montre un REVOKE. Aucun mot de passe réel n'est versionné : le secret applicatif est généré à l'installation dans un fichier exclu de Git.

Fonctionnalités du site et des quais

Chaque fonctionnalité est présentée sous la forme objectif, travail réalisé, conclusion. Les captures proviennent de l'application sur le jeu de données fictif.

1. Connexion et profils.

Objectif. Identifier les utilisateurs et limiter leurs droits selon leur profil.

Travail réalisé. Une inscription crée un compte en attente et inactif ; l'administrateur l'active et choisit son profil. La connexion vérifie le mot de passe haché et renouvelle la session. Les autorisations sont contrôlées à l'affichage des pages et à chaque action ; un transporteur est rattaché à une société.

Conclusion. Un compte en attente ou désactivé est refusé ; chaque profil ne voit que ses écrans. L'administrateur ne gère que les comptes et le journal.

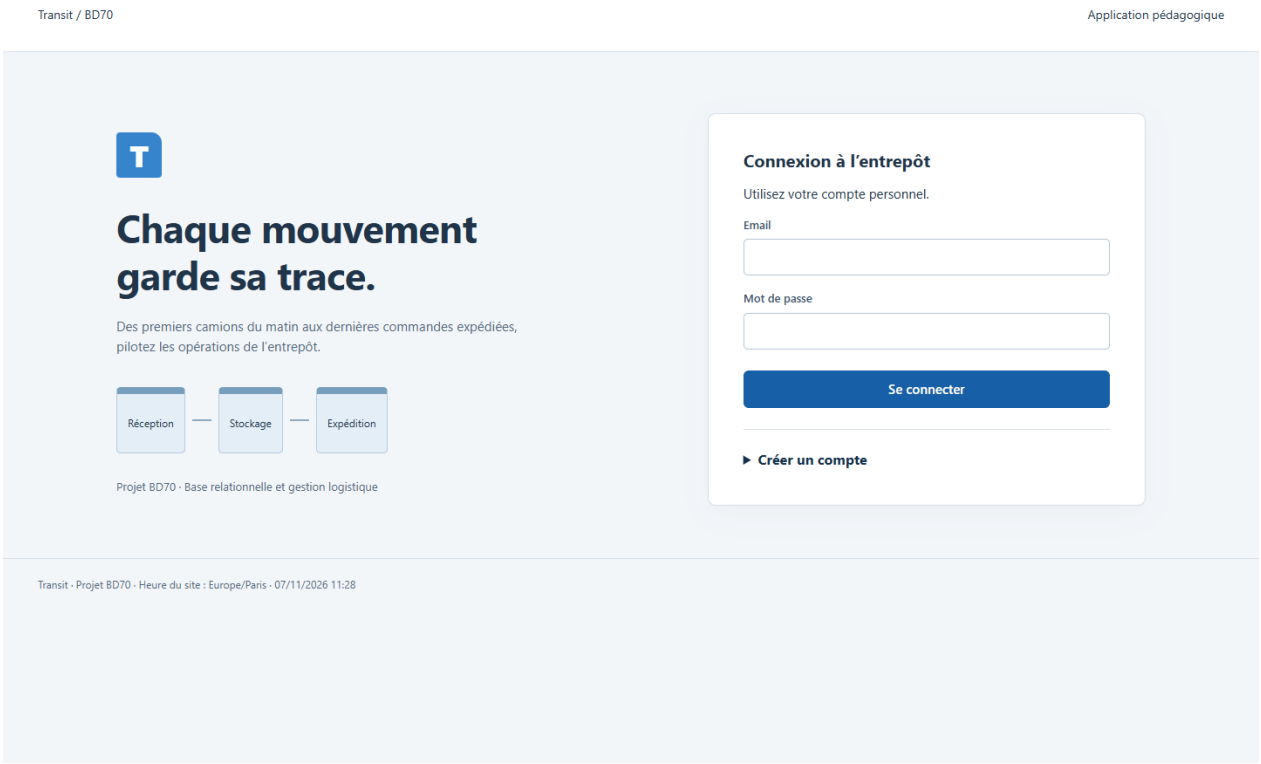


Figure 9 – Connexion et inscription.

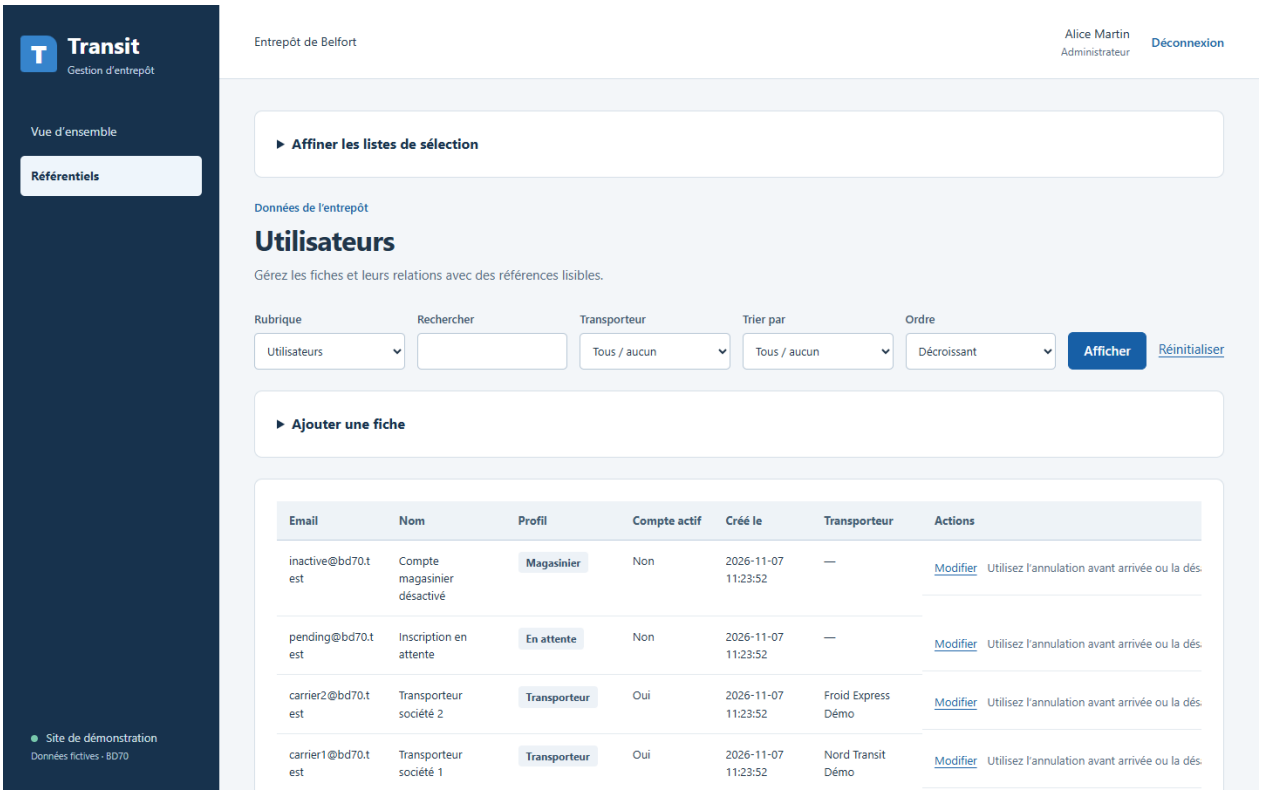


Figure 10 – Gestion des comptes par l'administrateur.

2. Transporteurs, chauffeurs et véhicules.

Objectif. Savoir qui transporte la marchandise et avec quel matériel.

Travail réalisé. La rubrique Référentiels permet d'ajouter, modifier, rechercher et supprimer sociétés, contacts, contrats, chauffeurs, habilitations, véhicules et équipements. Les listes déroulantes affichent des libellés lisibles (immatriculation, nom) et jamais les identifiants internes.

Conclusion. Une fiche inutilisée peut être supprimée ; une ressource déjà utilisée par un rendez-vous ou un passage est protégée, et ses caractéristiques structurantes sont figées.

3. Quais et rendez-vous.

Objectif. Attribuer un créneau réalisable et connaître l'état des quais.

Travail réalisé. Le responsable d'exploitation choisit un quai, un horaire et des ressources. Le serveur vérifie le type de quai, la compatibilité du véhicule et de ses équipements, l'appartenance du chauffeur et du véhicule à la même société, les indisponibilités et les chevauchements ; la base refuse elle-même tout chevauchement. Le transporteur gère les rendez-vous de sa société depuis son espace.

Conclusion. Le planning affiche chaque quai libre, réservé, occupé ou hors service et s'actualise toutes les trente secondes en dehors d'une saisie.

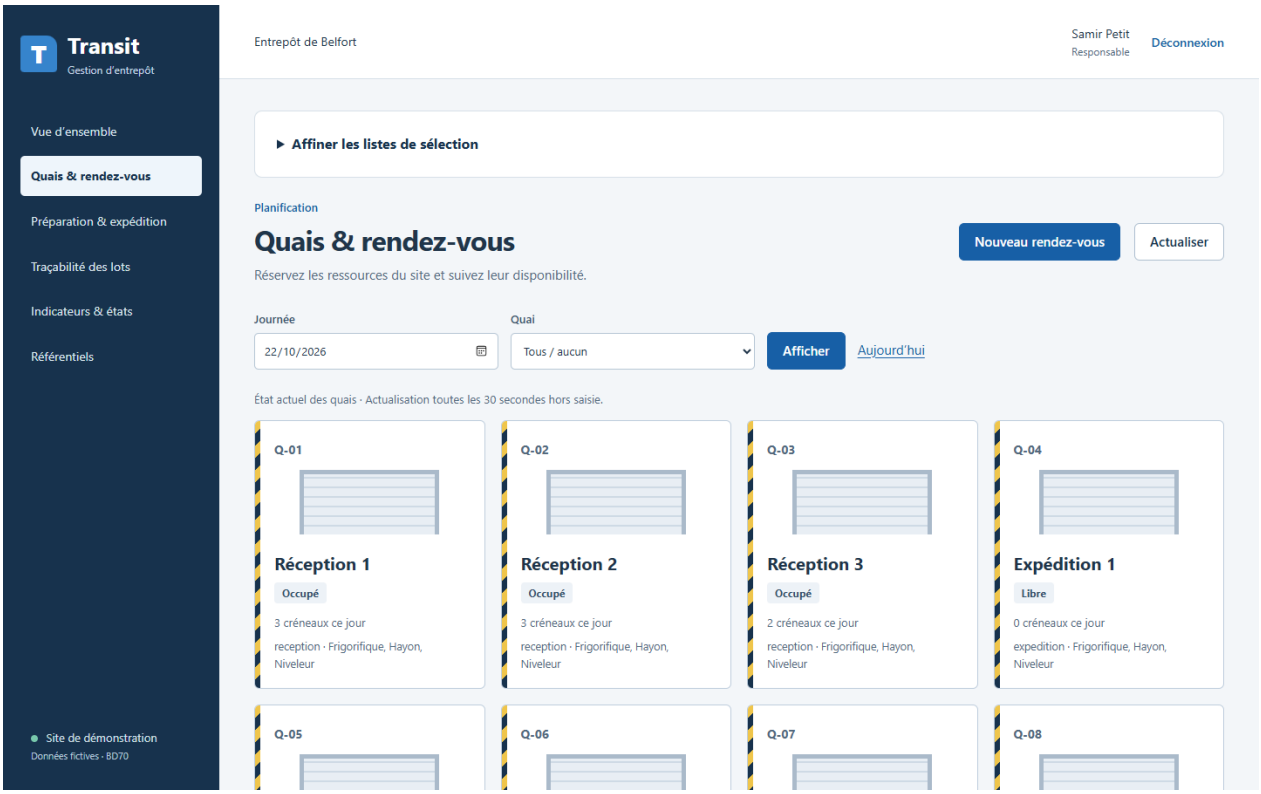


Figure 11 – Planning des quais vu par le responsable d'exploitation.

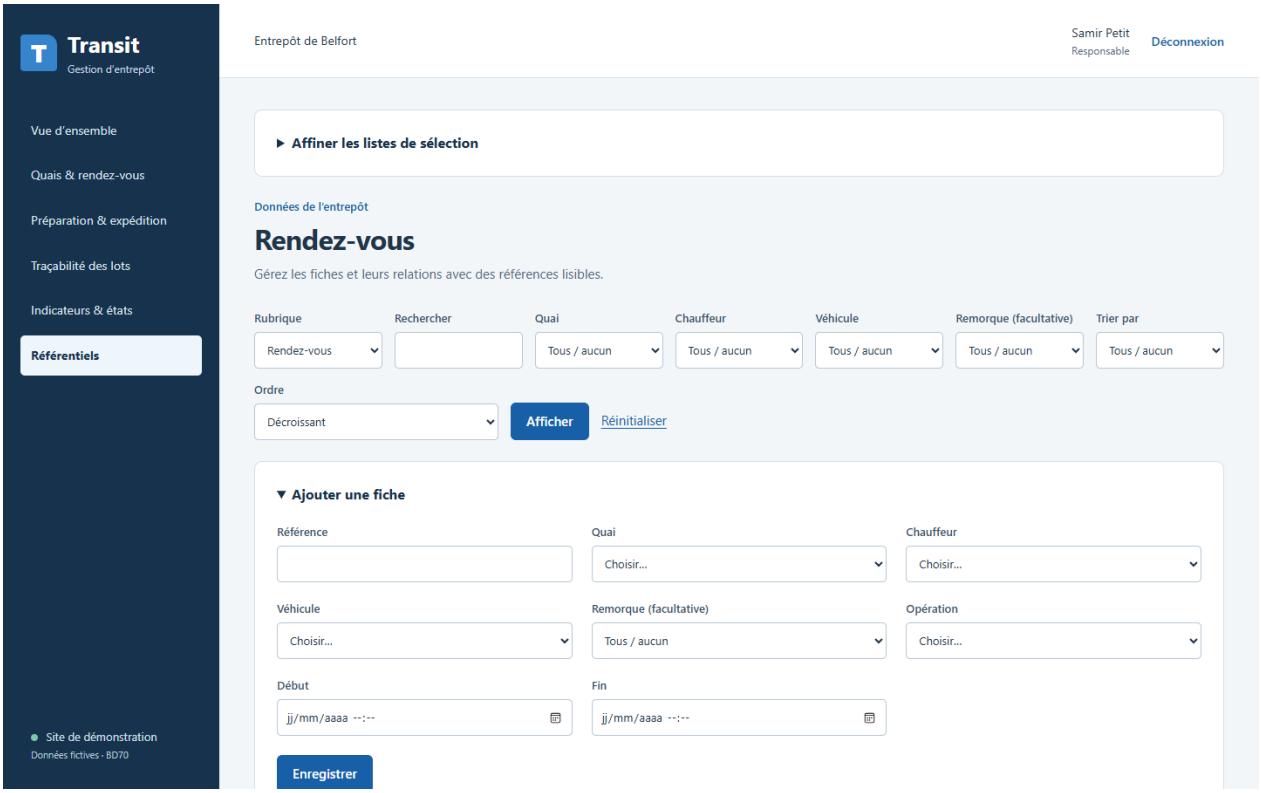


Figure 12 – Formulaire de rendez-vous.

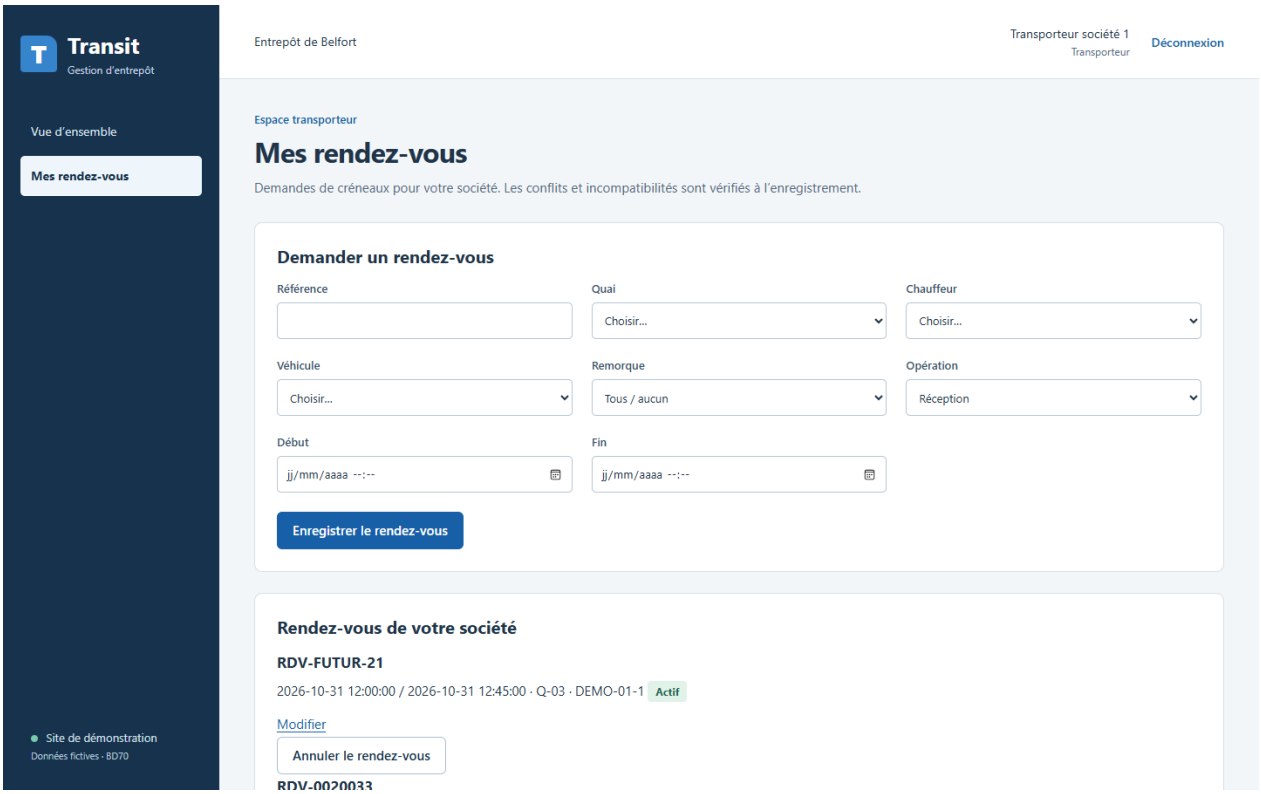


Figure 13 – Espace transporteur : rendez-vous de sa société.

4. Suivi des flux de camions.

Objectif. Mesurer où le camion passe du temps et détecter les retards.

Travail réalisé. L'accueil enregistre l'arrivée (avec ou sans rendez-vous), le contrôle documentaire, le parking facultatif, l'appel à quai et le départ ; le magasinier horodate le début et la fin de l'opération. Chaque étape est un événement immuable horodaté.

Conclusion. La vue v_visit calcule l'attente (arrivée → début d'opération) et l'immobilisation (arrivée → départ) ; l'écran compare l'arrivée au rendez-vous et filtre par période, transporteur et état.

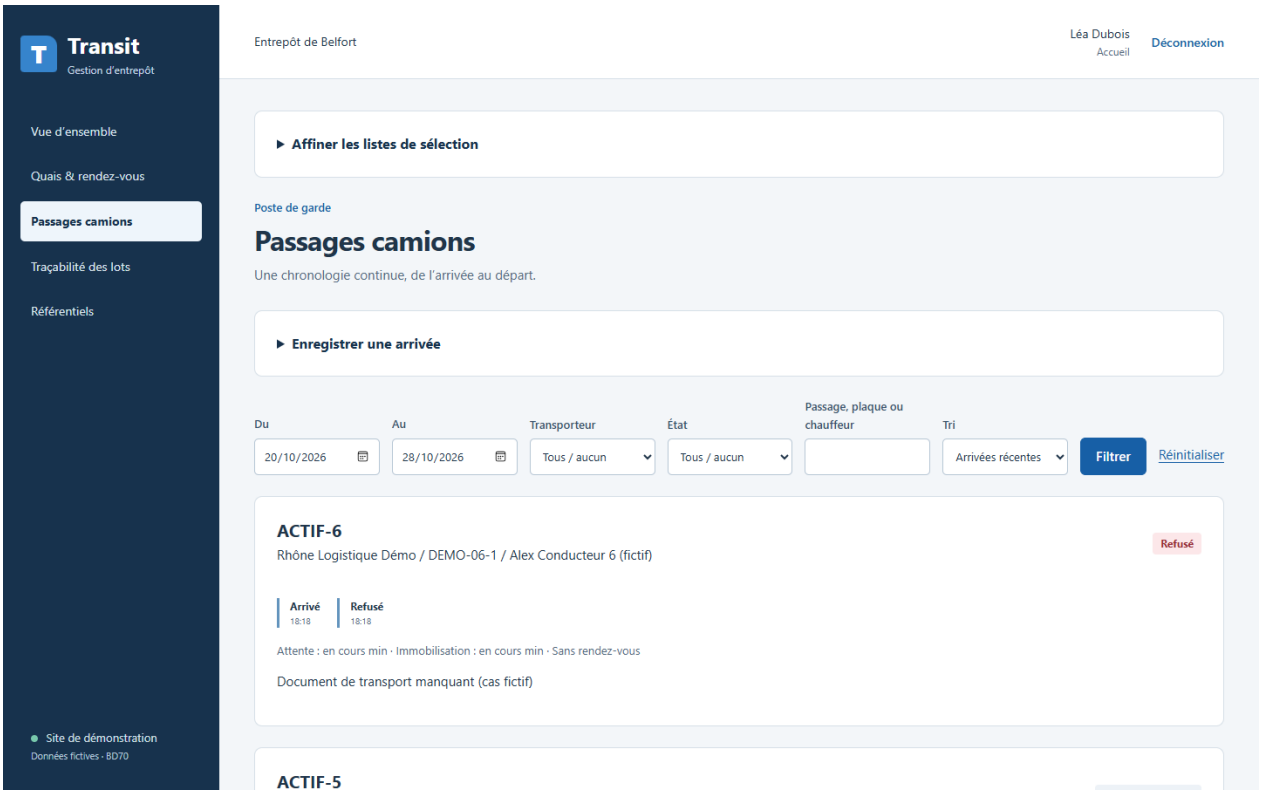


Figure 14 – Passages de camions, recherche multicritère.

Fonctionnalités de réception, de stock et d’expédition

5. Réception des marchandises.

Objectif. Comparer la livraison à l’avis du fournisseur.

Travail réalisé. Le magasinier ouvre une réception pour un camion à quai, saisit pour chaque ligne les fractions reçues : lot, date limite, quantités acceptée et endommagée, état de palette, réserve, emplacement. La validation contrôle les capacités et la compatibilité de zone, puis crée les mouvements d’entrée dans une seule transaction.

Conclusion. Seules les quantités acceptées entrent en stock ; les manquants, excédents et avaries sont calculés par vue et les réserves sont enregistrées, même pour un manque total.

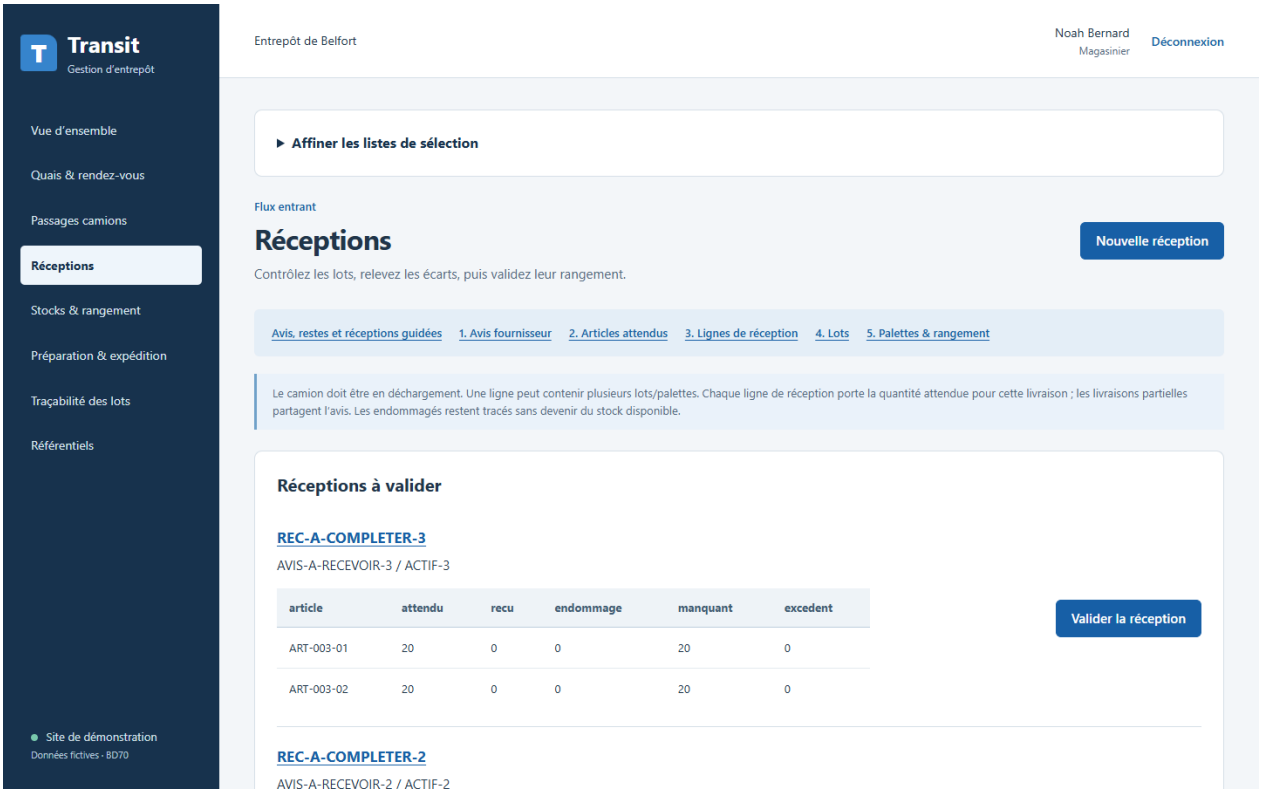


Figure 15 – Liste des réceptions et filtres.

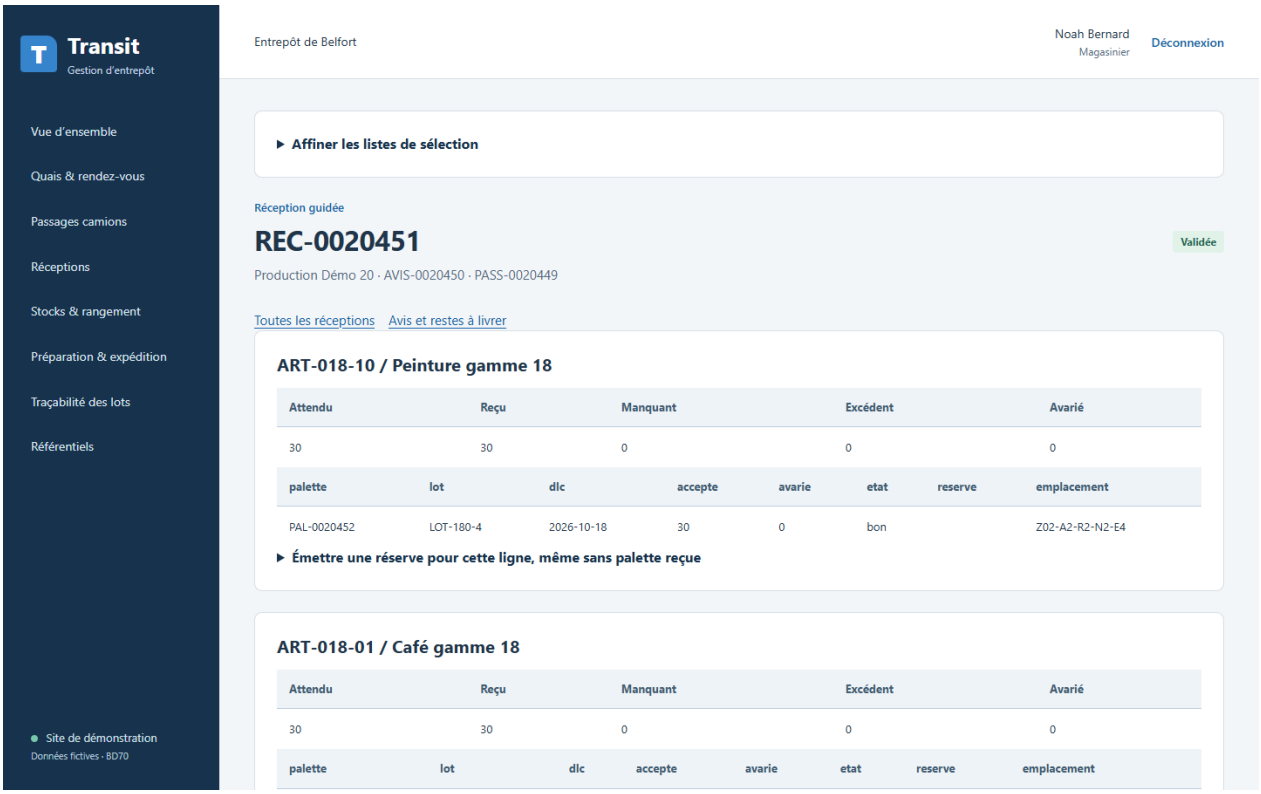


Figure 16 – Détail d'une réception validée.

6. Emplacements et stocks.

Objectif. Connaître la quantité d'un lot et sa position, et ranger au bon endroit.

Travail réalisé. La structure zone, allée, rack, niveau, emplacement porte les capacités. Les propositions de rangement tiennent compte du poids, du volume, des équivalents palettes et de la condition de stockage. Entrées, transferts, sorties et ajustements d'inventaire alimentent le journal des mouvements.

Conclusion. Le solde est calculé depuis ce journal ; chaque déplacement reste explicable. La recherche filtre par article, propriétaire, zone, lot ou emplacement et s'exporte en CSV.

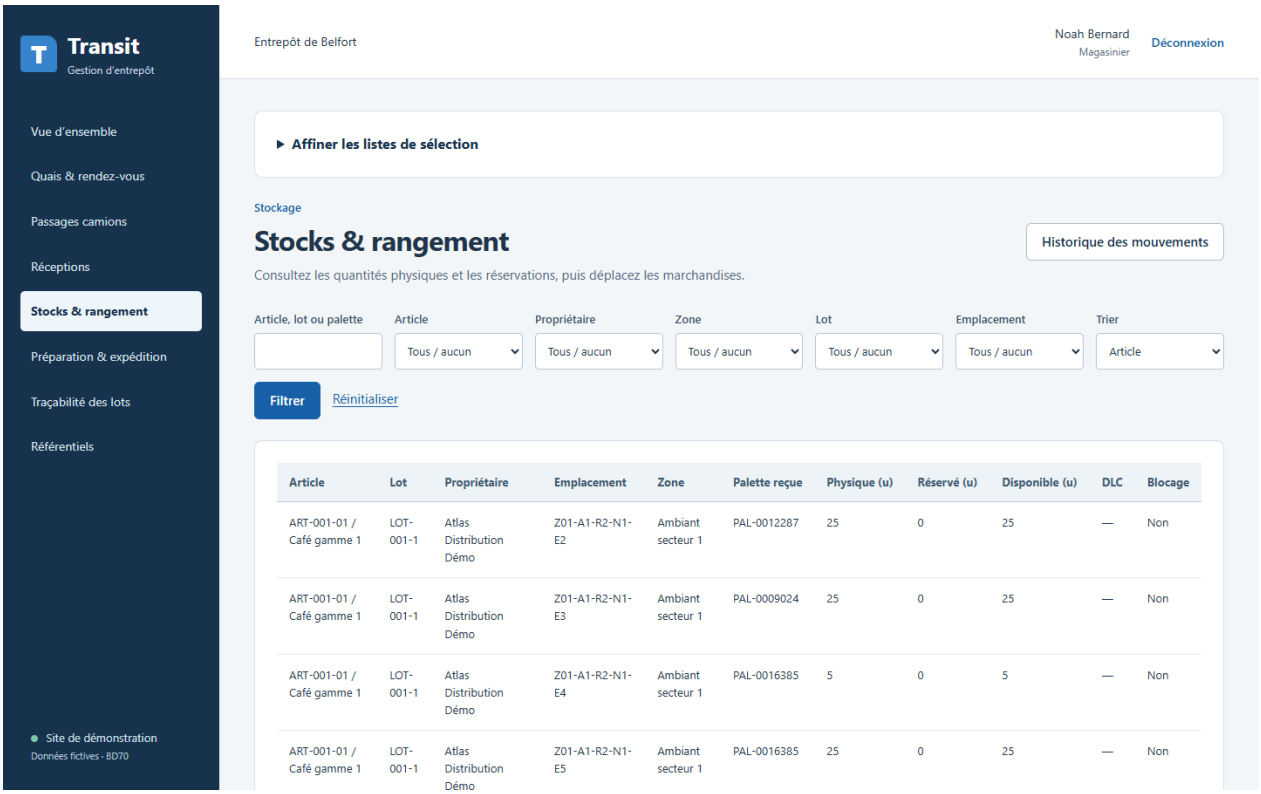


Figure 17 – État des stocks.

7. Préparation et expédition.

Objectif. Servir la commande avec les lots utilisables et charger les quantités prévues.

Travail réalisé. Le responsable crée la commande et l'affecte à un préparateur. Celui-ci lance la sélection FEFO (premier périmé, premier sorti), confirme les prélèvements, constitue colis et palettes, puis rattache la commande à un camion sortant.

Conclusion. La validation contrôle les quantités et la capacité du camion, crée les sorties de stock et produit le bon de livraison et la lettre de voiture imprimables.

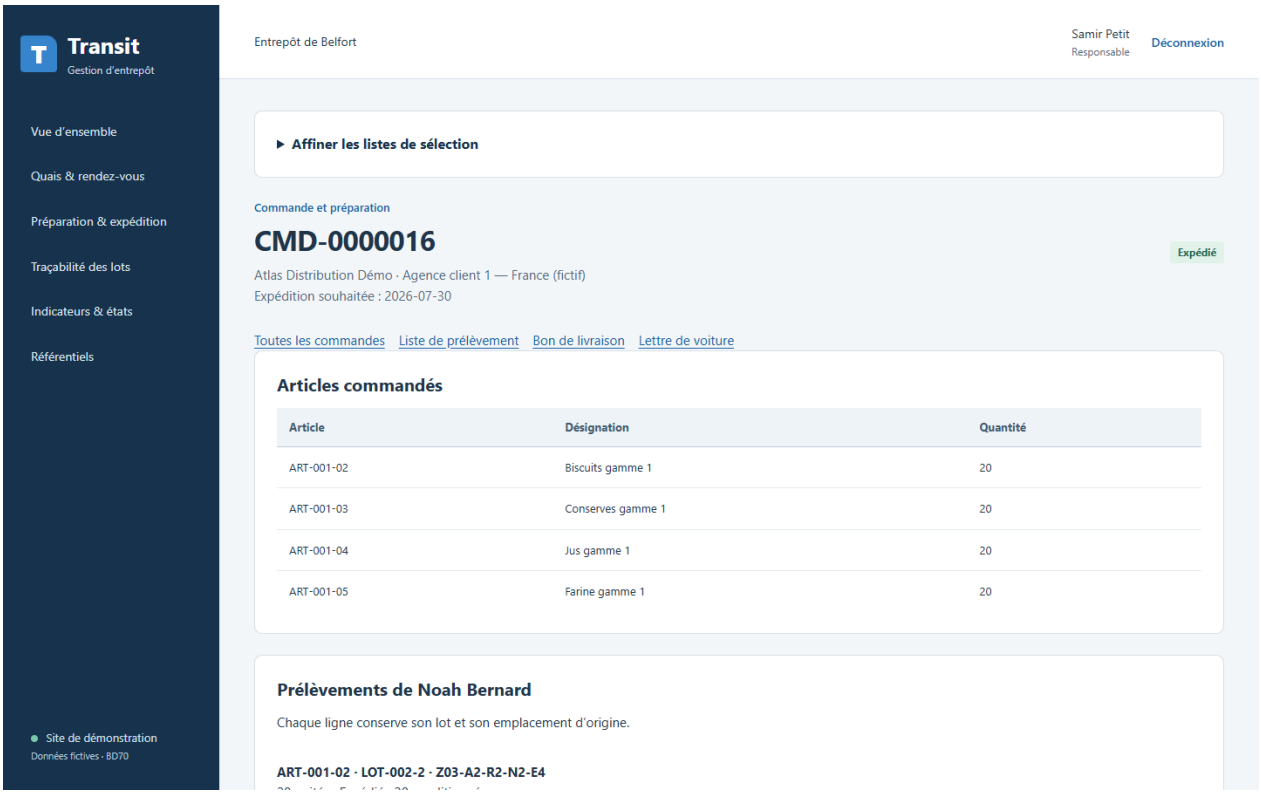


Figure 18 – Détail d'une commande expédiée.

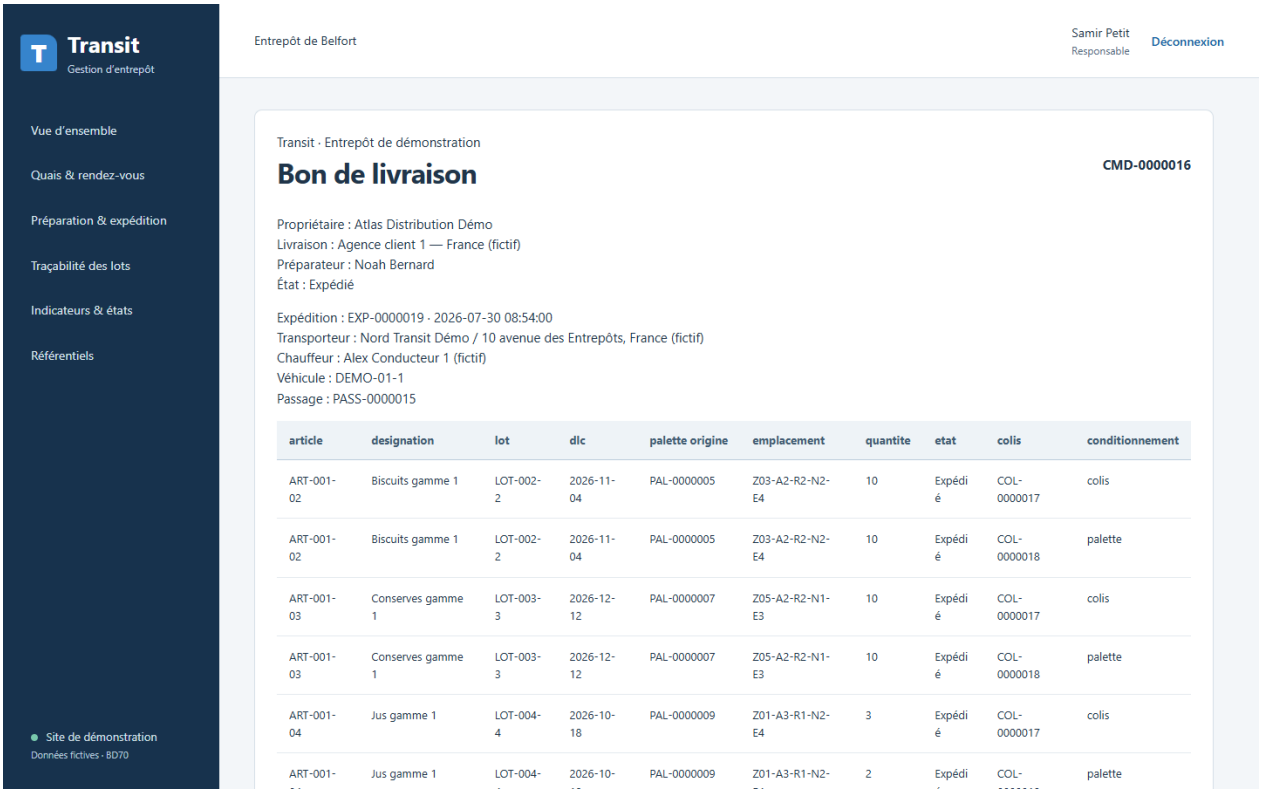


Figure 19 – Bon de livraison généré.

8. Traçabilité et tableaux de bord.

Objectif. Retrouver un lot et mesurer l'activité du site.

Travail réalisé. La recherche par lot appelle la procédure trace_lot et relie réceptions, mouvements, commandes et camions. Le tableau de bord calcule l'occupation des quais, les temps par transporteur, les transporteurs au-dessus d'un seuil d'attente (GROUP BY … HAVING), le remplissage par zone, les écarts en réception, la productivité des préparateurs et les lots proches de leur date limite, avec tableaux, graphiques, impression et

exports CSV.

Conclusion. À partir d'un numéro de lot, on retrouve la livraison d'origine et le camion de sortie ; les états sont filtrables par période.

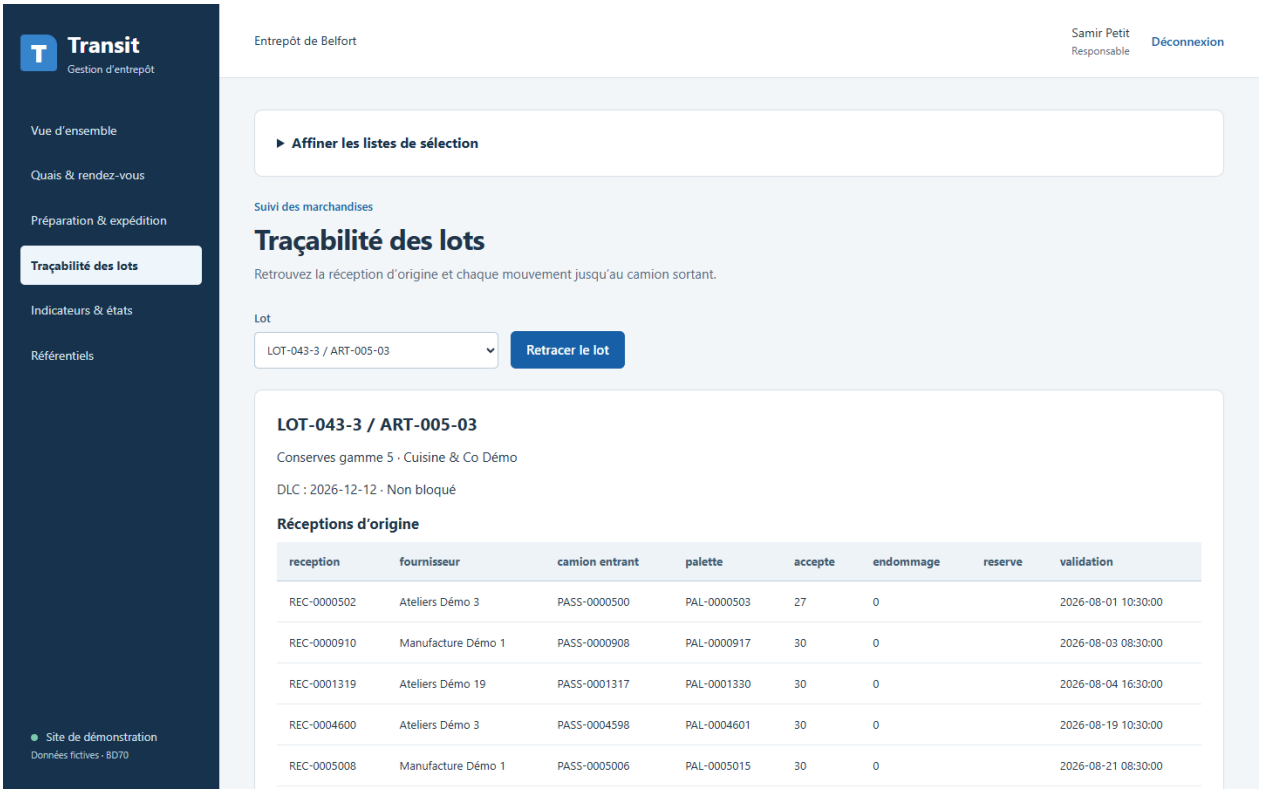


Figure 20 – Traçabilité d'un lot.

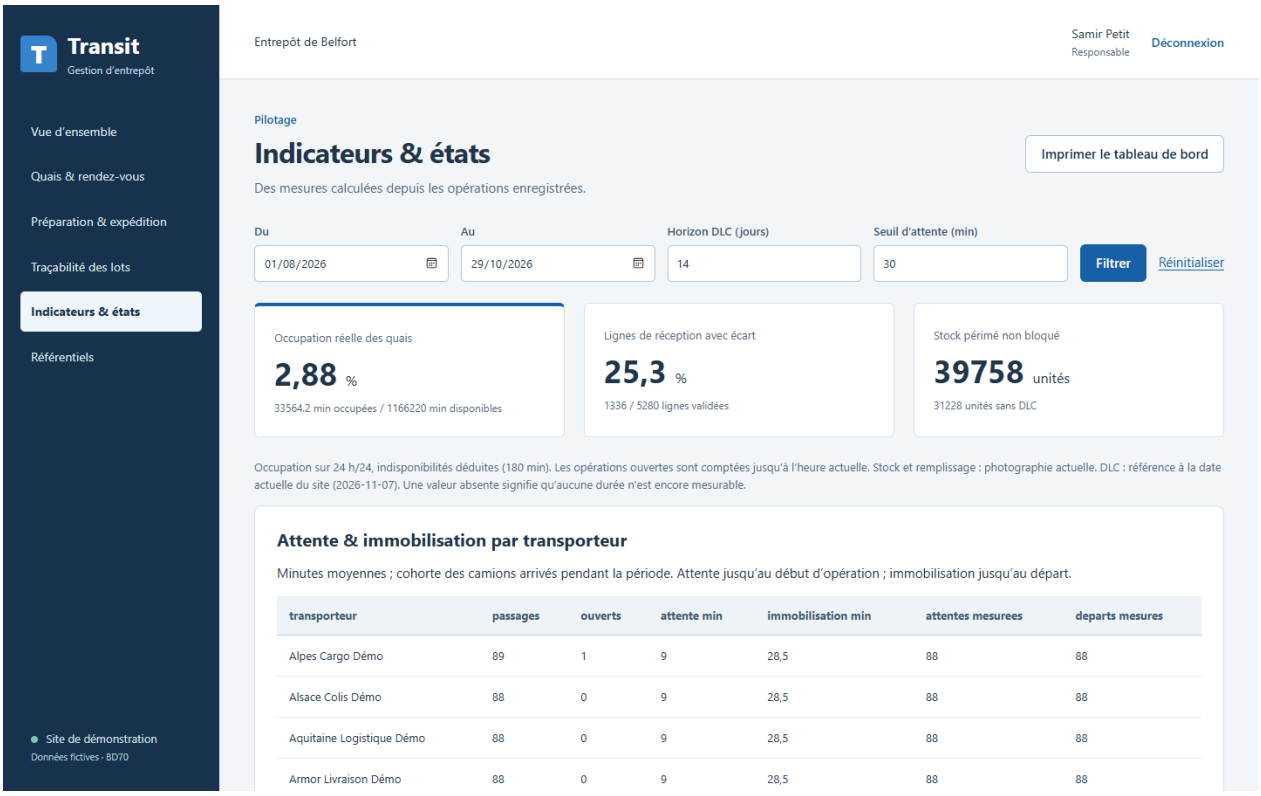


Figure 21 – Indicateurs et graphiques.

Transporteurs au-dessus du seuil d'attente

Attente moyenne supérieure à 8 min sur la période (regroupement par transporteur puis filtre HAVING). Aucune ligne : tous les transporteurs respectent le seuil.

transporteur	passages mesures	attente moyenne min	attente max min
Alpes Cargo Démo	88	9	12
Atlantique Transport Démo	88	9	12
Rhône Logistique Démo	88	9	12
Loire Distribution Démo	88	9	12
Bourgogne Fret Démo	88	9	12
Armor Livraison Démo	88	9	12
Provence Routier Démo	88	9	12
Alsace Colis Démo	88	9	12
Picardie Express Démo	88	9	12
Occitanie Cargo Démo	88	9	12
Normandie Transport Démo	88	9	12
Champagne Fret Démo	88	9	12
Jura Distribution Démo	88	9	12
Bretagne Routes Démo	88	9	12
Centre Palettes Démo	88	9	12
Aquitaine Logistique Démo	88	9	12
Vosges Transport Démo	88	9	12
Nord Transit Démo	89	8,9	12
Froid Express Démo	89	8,9	12
Route Est Démo	89	8,9	12

Figure 22 – Requête avec HAVING : transporteurs au-dessus du seuil d'attente.

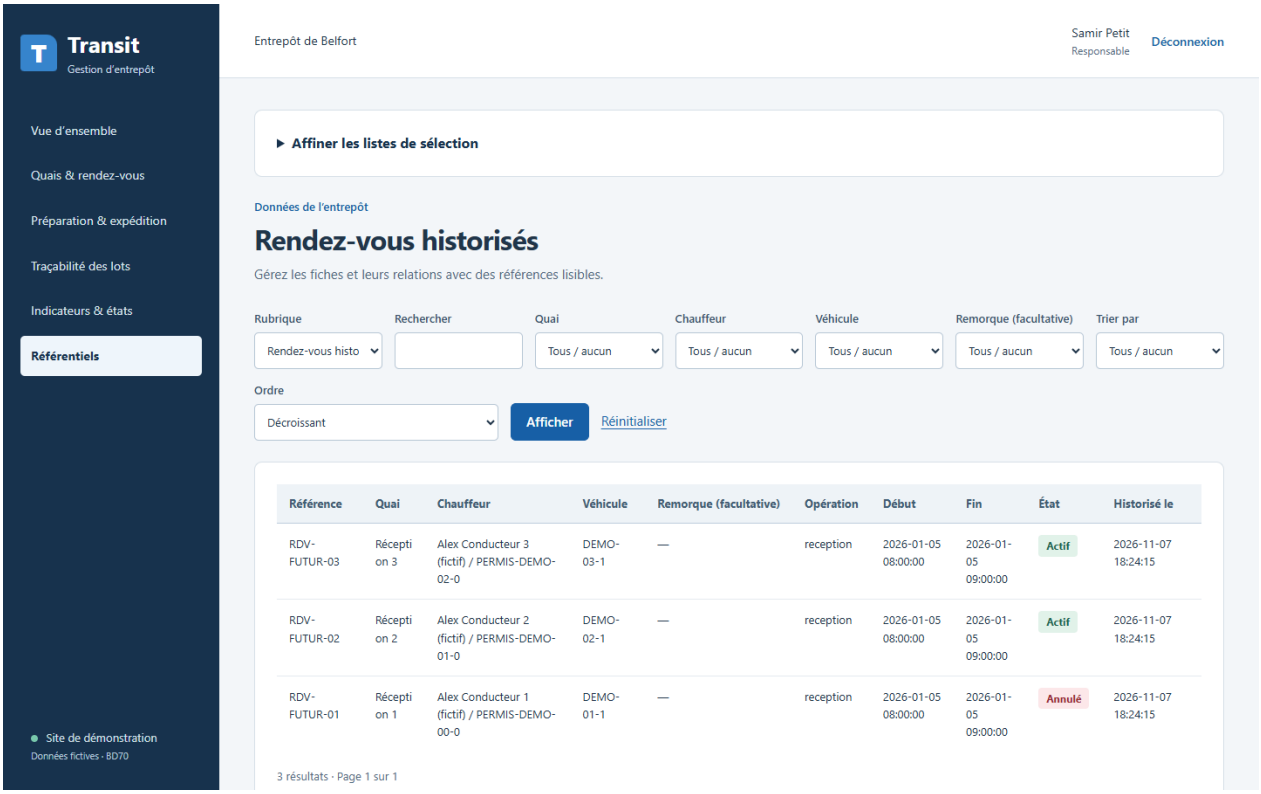


Figure 23 – Rendez-vous historisés (consultation en lecture seule).

Vérification

Les contrôles sont exécutés sur des bases de test neuves, jamais sur la base de démonstration. Le 7 novembre 2026, après les dernières modifications, nous avons obtenu les résultats suivants :

- les scripts SQL s’exécutent sans erreur ni avertissement sur une base vide, et les données factices se chargent intégralement ;
- 136 contrôles automatisés réussissent : 64 tests d’intégration, 7 tests de rapports, 19 tests de droits, 23 tests de complétude, 14 tests de finalisation et 9 tests d’optimisation (soldes égaux au recalcul après réservation, annulation et transfert, fonction stock_disponible, prélèvement expédié figé, soldes non modifiables par l’application) ;
- le déclencheur anti-chevauchement refuse un rendez-vous en conflit inséré directement en SQL ;
- l’archivage a déplacé trois rendez-vous anciens sans passage vers l’historique, la vue UNION ALL a conservé le même total et le journal a enregistré les suppressions.

Ces tests couvrent les cas nominaux et les refus : conflit de créneaux, dépassement de capacité, accès aux données d’une autre société, double validation, accès concurrents. Le scénario de soutenance (lot ORAL-CAFE-LOT : 20 unités annoncées, 18 acceptées et 2 endommagées, transfert de 3 unités, commande de 10 unités, expédition, traçabilité) est décrit pas à pas dans le guide illustré.

Données de démonstration et performances

Le jeu de démonstration est fictif et reproductible. Il couvre 90 jours simulés : 180 articles, 720 lots, 360 emplacements, 1 806 passages de camions, 903 réceptions, 924 commandes, 900 expéditions et environ 25 000 mouvements de stock. Une seconde base, de 10 000 passages, sert aux mesures de volume.

Chaque requête est exécutée une fois pour chauffer le cache, puis dix fois depuis PHP avec récupération des résultats ; le tableau donne les médianes. Les plans EXPLAIN, les paramètres et les valeurs extrêmes sont dans docs/performances.md et dans les preuves JSON.

Requête	Base de 10 000 passages (ms)	Jeu de démonstration (ms)
Disponibilité d'un quai	0,94	1,21
Recherche de passages	67,86	2,24
Recherche de stock	31,89	2,11
Emplacements compatibles	16,65	22,27
Historique d'un lot	51,33	52,73
Sélection FEFO	54,67	1,75
Statistiques par transporteur (GROUP BY)	62,46	25,40
Transporteurs au-dessus du seuil (HAVING)	58,25	17,12
Rendez-vous courants et historisés (UNION ALL)	31,03	3,52
Remplissage par zone	19,97	36,10

Démarche d'optimisation (TD Optimisation). Mesurer, lire le plan EXPLAIN, optimiser, puis mesurer à nouveau sur les mêmes données et calculer le gain absolu et relatif.

1. Index composé sur les créneaux. idx_appointment_slot(dock_id, status, starts_at, ends_at) fait passer la recherche de créneaux de 7,22 ms à 1,41 ms (gain absolu 5,81 ms, gain relatif 81 %) : EXPLAIN montre un accès par plage (range) au lieu de la lecture de tous les rendez-vous du quai.

2. Table de soldes tenue à jour par déclencheurs. Avant, la vue v_balance recalculait chaque solde par GROUP BY sur tout l'historique des mouvements, et v_reserved sur tous les prélèvements. EXPLAIN montrait des tables dérivées lues en entier (ALL) à chaque recherche de stock, sélection FEFO ou proposition d'emplacement : un coût qui croît avec l'historique. Après, stock_balance garde le solde et les réservations de chaque fraction par emplacement ; les déclencheurs le mettent à jour à chaque mouvement ou réservation, dans la même transaction ; v_balance et v_reserved lisent cette table par sa clé primaire et sont fusionnées dans la requête (plus de table dérivée). Mesures sur le jeu de démonstration, mêmes données, avant puis après (scripts/performance-optimisation.php) :

Requête	Avant (ms)	Après (ms)	Gain absolu (ms)	Gain relatif
Recherche de stock	72,29	1,99	70,30	97 %
Sélection FEFO	75,12	1,98	73,14	97 %
Emplacements compatibles	96,24	25,45	70,78	74 %
Remplissage par zone	101,39	19,98	81,41	80 %
Contrôle du stock libre (transfert)	63,81	0,38	63,43	99 %

Contrepartie : chaque écriture de mouvement exécute un déclencheur de plus ; l'écart mesuré (5,22 ms avant, 5,86 ms après) reste dans le bruit de mesure. Le solde devient une redondance contrôlée : l'historique immuable reste la référence et scripts/validate-data.php vérifie que chaque solde est égal à son recalcul.

Conclusion

Transit couvre les huit domaines du sujet avec une interface web, au-delà des cinq modules obligatoires. À partir d'un numéro de lot, l'application retrouve la livraison d'origine, les emplacements successifs et le camion d'expédition. Cette réponse repose sur trois décisions de conception : séparer l'attendu du réalisé, conserver la provenance des fractions et rattacher chaque sortie à un prélèvement identifié.

La base respecte la troisième forme normale, à quatre dénormalisations près, justifiées. Elle met en œuvre les notions du cours : clés et contraintes d'intégrité, jointures internes et externes, agrégations avec GROUP BY et HAVING, vues, procédures, fonction stockée, déclencheurs de cohérence, d'historique et de calcul incrémental, évènement planifié, transactions, index et optimisations mesurés avant et après, et droits limités.

Les limites sont connues : un seul site, une expédition par commande, et un verrou global qui simplifie la cohérence mais sérialise les écritures. Pour étendre le système, il faudrait des verrous par ressource et des expéditions partielles.

Le dossier livrables contient le diagramme de cas d'utilisation, le modèle de Chen et le schéma relationnel, chacun en PDF et SVG vectoriels et en source Draw.io modifiable, ainsi que les supports de soutenance. Le README explique comment installer Transit et charger le jeu fictif.